

Lumina Chronica

SUMMARY

Teil 3 – Technical Architecture Specification

ARCHITECTURE OVERVIEW

Architectural Goal: Lumina Chronica soll eine moderne, kostengünstige und skalierbare Webanwendung sein. Anforderungen: möglichst kostenlose Infrastruktur; einfache Wartung; einfache Erweiterbarkeit; geringe Betriebskosten; gute Performance; zukünftige Skalierbarkeit. Final Architecture Decision: Static Frontend + Serverless Backend Architecture.

REPOSITORY STRUCTURE

Repository Model: Monorepo. Grundstruktur: frontend/, backend/, shared/, documentation/, database/, scripts/, tests/, README.md. Grund: Entwicklung, Dokumentation, Versionierung, Deployment, Zusammenarbeit.

FRONTEND ARCHITECTURE

Technology Decision: Blazor WebAssembly. Frontend Hosting: GitHub Pages (Git Push → GitHub Actions → Build Blazor WASM → Deploy GitHub Pages). Responsibilities: UI Darstellung, Navigation, Benutzerinteraktion, Reader Interface, lokale Zustände, API Kommunikation. Nicht enthalten: direkte Datenbankzugriffe, geheime Schlüssel, sensible Logik. Empfohlene Struktur mit Pages, Components, Services, Models, Styles.

BACKEND ARCHITECTURE

Technology Decision: Cloudflare Workers. Responsibilities: Authentifizierung, Autorisierung, API, Datenzugriff, Upload-Verarbeitung, Business Logic. Struktur: routes (auth.ts, books.ts, users.ts, projects.ts, statistics.ts), services, middleware, models, utils.

DATABASE & STORAGE ARCHITECTURE

Database: Cloudflare D1 (SQLite kompatibel, serverless, günstig). D1 speichert Benutzer, Bücher-Metadaten, Beziehungen, Fortschritte, Projekte, Einstellungen; nicht große Dateien, EPUB/PDF, Bilder. Storage: Cloudflare R2 für Buchdateien, Cover, Bilder, Projektdateien, Karten. Beispielstruktur für books/ und projects/.

AUTHENTICATION & API ARCHITECTURE

Goal: Sichere Benutzerverwaltung. Possible Solutions Priorität: Cloudflare Access/Auth Integration, OAuth Provider, eigenes Auth System. Authentication Flow: User Login → Provider → Token → Frontend session → API validates token → Access granted. API Style: REST API. Base: /api/. Endpoints für auth, books, reading progress, projects.

SCALABILITY, PERFORMANCE, SECURITY

Initial Target v1: 100–1000 Bücher, 100–500 Nutzer. Future Target: 10.000+ Nutzer, 100.000+ Bücher. Performance: Frontend (Lazy Loading, Component Splitting, Caching, Pagination), Backend (effiziente Queries, API Pagination, Response Caching), Storage (direkte Datei-URLs, CDN Nutzung, Komprimierung). Security: niemals Passwörter speichern, private Dateien nicht öffentlich, Eingaben validieren, Rechte prüfen. Permission System: Private, Shared, Public.

DEVELOPMENT, DEPLOYMENT, TESTING

Required Tools: .NET SDK; Visual Studio/Rider/VS Code; Node.js; Cloudflare Wrangler; SQLite Tools; Git; GitHub. Development Workflow: Feature Idee → Issue → Planung → Development Branch → Testing → Pull Request → Review → Merge. Deployment: Frontend via GitHub Actions to Pages; Backend via Cloudflare Deployment/Worker Update. Testing Strategy: Frontend (Komponenten, Navigation, Reader), Backend (API, Auth, Datenbank), User Tests mit echten Nutzern.

FUTURE EXTENSIONS & SUMMARY

Mögliche Erweiterungen: Mobile App (.NET MAUI, React Native, Flutter), Search Engine (eigene Suchindizes, Volltextsuche), AI Services (separates AI Backend). Architecture Summary: USER → Blazor WebAssembly → GitHub Pages → Cloudflare Workers API → D1/R2/Auth. Architekturprinzip: Einfach starten, professionell skalierbar bleiben.